

Parallelising a Computational Fluid Dynamics Solver using Distributed Memory Parallelisation

Sam Armstrong

Abstract—Computational Fluid Dynamics simulations based on the Navier-Stokes equations are essential for modelling complex flows but are computationally expensive when executed serially. This paper presents the parallelisation of the Karman solver, a 2D incompressible flow application, using the MPI message passing interface for distributed memory parallelisation and the OpenMP API for shared-memory parallelisation; together forming a hybrid approach. We demonstrate the capability of this architecture to scale both across processes and threads of a single node, achieving near-linear scalability in this context, and across multiple nodes, achieving good scalability and performance - though with diminishing performance returns as the number of nodes increases. We suggest the reasoning for this, namely the communication overhead between nodes, choosing a non-blocking communication strategy to help mitigate it, and suggest future optimisations to help address it.

I. INTRODUCTION

Computational Fluid Dynamics is important in modern engineering and scientific research for enabling the simulation of complex fluid flows without the cost of physical experimentation. The fundamental governing rules of these simulations are the Navier-Stokes equations, which describe the motion of viscous fluid substances [1]. However, solving these equations numerically over high-resolution discretised domains is computationally expensive, necessitating the use of High-Performance Computing techniques to make them practically feasible. This project focuses on the parallelisation of Karman a sequential Computational Fluid Dynamics solver designed to model incompressible flow in two dimensions [1].

The objective of this project is to accelerate the Karman solver first using MPI [2], a message passing interface that defines a standard for how nodes with different memory spaces can communicate, which we'll use to parallelise the application across nodes that have no shared memory. Parallelisation with OpenMP, which is a standard API for shared-memory parallel programming [3], will then be applied on top of this to improve per-node performance, as we can leverage the shared memory space of each node to reduce the communication overhead and apply optimisations like vectorisation.

We begin by implementing a domain decomposition strategy that is used to divide the computational grid among MPI ranks, and the I/O strategy used to gather the data from the individual ranks into a single output file. We then explore the performance of the application using MPI compared to the unparallelised application, and evaluate how well it scales with problem size and number of ranks, as well as how well it uses the system resources (system peak performance). We'll

then explore how OpenMP can be implemented into this parallelised application, and assess how it impacts the throughput, bandwidth utilisation and system peak performance.

II. PARALLELISING WITH MPI

A. Domain Decomposition

The first step in parallelising the Karman solver with MPI is to define the domain decomposition strategy we'll use to divide the computational grid among the MPI ranks - the processes which will each process a sub-domain of the grid. For this, we'll use a one dimensional decomposition by splitting the domain in the y-direction. This is better than splitting over the x-direction as in C, two dimensional arrays are stored such that rows are contiguous in memory but columns are strided, meaning accessing rows in the memory should be theoretically faster than columns.

As each point in the grid needs to be updated using the five-point stencil from the neighbouring points, we also need to allocate extra rows to each process that are necessary for the computation, but do not need to be updated themselves - these are called halo cells. Additionally, at each time step the boundary values must be communicated to neighbouring ranks to keep the halo cells up to date. To communicate the halo cells between ranks, we have the choice of using blocking communications, which block the calling process until the communication is complete, or non-blocking communications, which allow the calling process to continue with its computation while the communication is ongoing. Blocking communications are easier to implement, but add overhead to the application, as it has to wait for the communication to complete before it can start processing the next timestep, which can be substantial as nodes need to communicate over a relatively slow network. Therefore, we use non-blocking communications, starting to process the cells that are not dependent on halo cells immediately, then waiting to process the cells dependent on halo cells until the communication is complete, and the halo cells are therefore up to date.

B. I/O Strategy

At the end of the parallel application, the individual domains that have been computed by the parallel ranks need to be assembled into a single grid, so it can be converted to the final karman.bin file. To do this, we gather data into the root process using blocking communications to ensure we have the full grid of data, before writing to the output file. The correctness of the output is verified by comparing the parallel and non-parallel solutions using diffbin, which produced identical results.

C. Performance and Scalability

The performance results of the application parallelised with MPI are shown in Table I. The application was tested against a variety of problem sizes, number of nodes, and number of processes; calculating the application runtime, throughput GFLOPs per node, and bandwidth utilised per node to help assess how the model is truly performing and making use of the available hardware resources. The application was tested on a cluster of Intel i5-8500 CPUs (each with 6 cores). The peak memory bandwidth of these is 41.6 GB/s [4] and the peak compute performance is 288 GFLOPs [5].

1) *Scaling on a Single Node*: The application appears to scale with increasing number of processes on a single node and increasing problem size. Using a single node with a single process on the 660x120 problem size achieves a throughput of 7.188 GFLOPs, which increases to 14.092 GFLOPs when using two processes, and 33.385 GFLOPs when using six processes. This represents near-linear scaling of performance with the number of processes on the same node. Likewise, when we scale up the problem size on the same node using six processes, matching the number of cores, we achieve 33.385 GFLOPs on a 660x120 problem size, 33.503 on 1320x240, and 30.919 on 1980x360; this shows the hardware utilisation remains relatively consistent as we scale up the problem size. This strong scaling on a single node is due to the low overhead of communication between processes, which is very low compared to the computation, and far lower than for communication between nodes. An interesting observation from my results is that increasing the number of processes above the number of cores significantly damages performance. This occurs because processes are competing for the same resources, so each process gets less time to execute.

2) *Scaling Across Nodes*: Scaling across nodes is working reasonably well, but there is a performance trade-off to using multiple nodes, so the results show it's only worth doing for larger problem sizes. For example, on the 660x120 problem size, the fastest runtime was when running six processes on a single node (3.60s), while running the same problem on two nodes with six processes each took a much longer 9.16s. On the 1980x360 problem size, by comparison, the fastest runtime was when running eight nodes with six processes each (87.82s), which was significantly better than running six processes on a single node (150.40s). However, there are diminishing marginal returns with using an increasing number of nodes, which is exemplified by the same run on six nodes taking 89.92s, which is very nearly as fast as running on eight nodes but only taking 75% of the resources.

These diminishing marginal returns are further shown, as when we scale up the number of nodes, the GFLOPs used per node decreases quite drastically - from 30.919 on one node all the way down to 6.621 on eight nodes, and similar is true for the bandwidth utilised per node. This worsening throughput and bandwidth usage is due to the overhead of communication between nodes increasing as the number of nodes increases, as they communicate over a relatively slow network rather than the internal system bus. This is time during which the

CPU cores could be idle, losing a significant amount of their theoretical performance. This shows the importance of the non-blocking communication methodology implemented, as it somewhat mitigates the impact of this long communication time between nodes by beginning the process of calculating the next time step without waiting for the communication to complete. This overhead of node communication is nevertheless an issue though, and we can see it in action when running a configuration of four nodes, each with one process, results in a runtime of 62.78s - compared to running the four processes on a single node, which takes 41.65s (problem size 1320x240). This shows the importance of using all the processes on a node, to get the best use out of the cores, before scaling up to multiple nodes. It's also worth noting that the arithmetic intensity drops somewhat as the number of nodes increases, suggesting the communication overhead is resulting in a higher proportion of memory based operations relative to compute based operations.

3) *Utilisation of System Peak Performance*: To help assess how well the application is making use of the system peak performance, we've plotted a roofline model of the application in Figure 1. As we can see from Table I, the highest throughput GFLOPs we achieve is 33.97, which is significantly below the quoted theoretical peak of 288 GFLOPs, which can be visualised by the large gap between the data points and the roofline (Figure 1). Likewise, the bandwidth utilised only reaches 0.859 GB/s, again far below the quoted peak of 41.6 GB/s. While this theoretical peak may have a disparity with the true attainable peak, these gaps are quite significant. We can see from the roofline model that all points lie within the compute-bound region, showing that the application is performing a lot of calculations per byte of memory loaded (high arithmetic intensity), but these are not being processed in a very efficient way. This shows that there is room for further optimisation of the application.

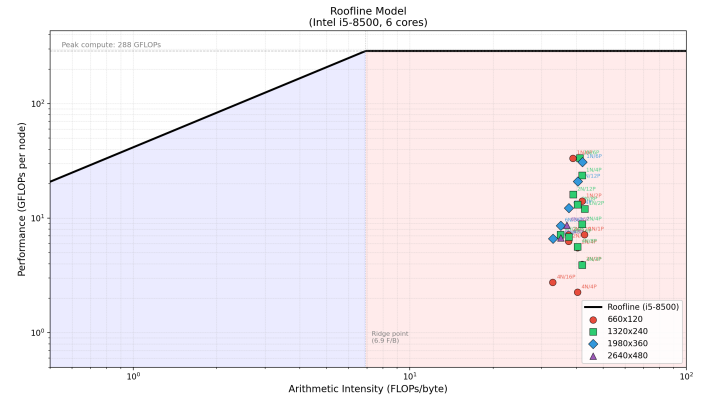


Fig. 1. Roofline model for Karman Parallelised with MPI

III. OPTIMISING WITH OPENMP

This section focusses on optimising the application per-rank with shared memory parallelism using OpenMP. Using a hybrid architecture like this allows us to leverage the benefits of

TABLE I
KARMAN (MPI) PERFORMANCE ACROSS PROBLEM SIZES AND NUMBER OF NODES AND PROCESSES

Problem Size	Nodes	Processes	Total Bandwidth	Total GFLOPs	Bandwidth per Node	GFLOPs per Node	Runtime	AI
660x120	1	1	0.168	7.188	0.168	7.188	16.14	42.81
660x120	1	2	0.336	14.092	0.336	14.092	8.24	41.96
660x120	1	6	0.859	33.385	0.859	33.385	3.6	38.884
660x120	2	2	0.188	7.893	0.094	3.946	14.7	41.96
660x120	2	4	0.275	11.09	0.138	5.545	10.57	40.365
660x120	2	8	0.334	12.54	0.167	6.27	9.45	37.508
660x120	2	12	0.409	14.322	0.204	7.161	9.16	37.508
660x120	4	4	0.224	9.03	0.056	2.258	13.1	40.365
660x120	4	16	0.335	11.02	0.084	2.755	10.8	32.859
1320x240	1	2	0.281	12.068	0.281	12.068	81.38	42.878
1320x240	1	4	0.561	23.581	0.561	23.581	41.65	42.03
1320x240	1	6	0.813	33.503	0.813	33.503	29.31	41.214
1320x240	2	4	0.422	17.72	0.211	8.86	55.42	42.03
1320x240	2	8	0.652	26.355	0.326	13.178	37.27	40.429
1320x240	2	12	0.826	32.181	0.413	16.091	30.52	38.946
1320x240	4	4	0.372	15.643	0.093	3.911	62.78	42.03
1320x240	4	8	0.557	22.538	0.139	5.635	43.58	40.429
1320x240	4	16	0.731	27.472	0.183	6.868	35.75	37.568
1320x240	4	24	0.82	28.78	0.205	7.195	34.13	35.086
1980x360	1	6	0.735	30.919	0.735	30.919	150.40	42.05
1980x360	2	12	1.035	41.878	0.518	20.939	111.05	40.448
1980x360	4	24	1.308	49.181	0.327	12.295	94.57	37.586
1980x360	6	36	1.474	51.73	0.246	8.622	89.92	35.103
1980x360	8	48	1.609	52.966	0.201	6.621	87.82	32.928
2640x480	6	36	1.417	52.337	0.236	8.723	280.67	36.943
2640x480	8	48	1.538	53.985	0.192	6.748	272.11	35.112

Bandwidth measured in GB/s; AI - Arithmetic Intensity measured in FLOPs/byte; Runtime measured in seconds.

both shared and distributed memory parallelism. For example, on an individual node we can use shared memory to avoid the communication overhead of things like exchanging halo cells between processes; but when we have a large problem, we can still efficiently scale it across multiple nodes using distributed memory parallelism.

A. Implementation

1) *Vectorisation and Reduction:* We used SIMD (Single Instruction, Multiple Data) parallelism in the `compute_rhs` function and the reduction loops in `poisson` and `set_timestep_interval`. By showing the compiler that loop iterations are independent and safe to vectorise, the CPU can process multiple floating point operations simultaneously, calculating the RHS for multiple grid cells in a single CPU cycle. In the `poisson` solver, combining `simd` with the reduction clause allowed the residual errors to be summed in a vectorised operation, removing a serial bottleneck from the application.

2) *Scheduling and Load Balancing:* For the velocity updates and RHS computation, where the workload per grid cell is uniform, we used static scheduling with the `#pragma omp schedule(static)` directive. This minimises scheduling overhead by dividing the loop iterations into fixed-size chunks at compile time. Inside the `poisson` solver's convergence loop, we instead used guided scheduling with the `#pragma omp schedule(guided)` directive, as the time taken for threads to complete their chunks can vary as the Red-Black SOR progresses. The guided schedule works well for this case because the computational load per row varies due to the presence of the obstacle in the grid. Rows containing the obstacle require fewer fluid updates than rows without,

creating a load imbalance that static scheduling fails to address efficiently.

Another factor that allows for better load balancing is that, for the velocity computation, the calculations for the horizontal and vertical velocity updates are independent within a single time step. We exploit this feature using the `nowait` clause in the `#pragma omp for` directive, which removes the implicit barrier at the end of the loop, allowing threads that finish their assigned updates early to immediately proceed to the next loop without waiting for slower threads. This smooths out load imbalances to keep the threads busy, improving the overall parallel efficiency of the `compute_tentative_velocity` and `update_velocity` functions.

B. Performance and Scalability

As with the solely MPI parallelised application, we tested against a variety of problem sizes, this time taking into account the number of nodes, the number of processes, and the number of OpenMP threads. The performance results of the application parallelised with the hybrid approach are shown in Table II.

The first thing to note is that the performance of the hybrid application is generally worse than the application solely parallelised with MPI for smaller problem sizes across few nodes, but the hybrid application is better when scaling up to larger problem sizes and running across multiple nodes. For example, for the 1320x240 problem size running on one node, the hybrid application takes 30.37s, while the MPI application only takes 29.31s; but for the 1980x360 problem size running on four nodes, the hybrid application takes 92.86s, while the MPI application only takes 94.57s. We can also see that for

TABLE II
KARMAN (MPI + OPENMP) PERFORMANCE ACROSS PROBLEM SIZES AND NUMBER OF NODES/PROCESSES/THREADS

Size	Nodes	Processes	OMP Threads	Total Bandwidth	Total GFLOPs	Bandwidth/Node	GFLOPs/Node	Runtime	AI
660x120	1	6	1	0.862	33.525	0.862	33.525	3.58	38.884
660x120	1	3	2	0.771	31.719	0.771	31.719	3.78	41.148
660x120	1	2	3	0.757	31.753	0.757	31.753	3.78	41.963
660x120	1	1	6	0.743	31.803	0.743	31.803	3.77	42.81
1320x240	1	6	1	0.785	32.332	0.785	32.332	30.37	41.214
1320x240	1	3	2	0.746	31.678	0.746	31.678	31.0	42.45
1320x240	1	2	3	0.736	31.546	0.736	31.546	31.13	42.878
1320x240	1	1	6	0.718	31.089	0.718	31.089	31.59	43.316
1320x240	2	12	1	0.84	32.707	0.42	16.353	30.03	38.946
1320x240	2	6	2	0.719	29.612	0.359	14.806	33.16	41.214
1320x240	2	4	3	0.733	30.794	0.366	15.397	31.89	42.03
1320x240	2	2	6	0.717	30.735	0.358	15.367	31.95	42.878
1980x360	1	6	1	0.715	30.052	0.715	30.052	154.74	42.05
1980x360	1	3	2	0.671	28.767	0.671	28.767	161.64	42.899
1980x360	1	2	3	0.69	29.814	0.69	29.814	155.97	43.19
1980x360	1	1	6	0.65	28.251	0.65	28.251	164.6	43.484
1980x360	2	12	1	1.028	41.567	0.514	20.783	111.88	40.448
1980x360	2	6	2	0.947	39.834	0.474	19.917	116.74	42.05
1980x360	2	4	3	0.978	41.682	0.489	20.841	111.56	42.612
1980x360	2	2	6	0.961	41.491	0.48	20.746	112.07	43.19
1980x360	4	24	1	1.281	48.163	0.32	12.041	96.56	37.586
1980x360	4	12	2	1.219	49.29	0.305	12.323	94.35	40.448
1980x360	4	8	3	1.207	50.08	0.302	12.52	92.86	41.502
1980x360	4	4	6	1.141	48.604	0.285	12.151	95.67	42.612
2640x480	4	24	1	1.26	49.093	0.315	12.273	299.19	38.975
2640x480	4	12	2	1.169	48.218	0.292	12.054	304.6	41.244
2640x480	4	8	3	1.182	49.7	0.295	12.425	295.51	42.061
2640x480	4	4	6	1.145	49.134	0.286	12.283	298.9	42.91

Bandwidth measured in GB/s; AI - Arithmetic Intensity measured in FLOPs/byte; Runtime measured in seconds.

smaller problem sizes, relying entirely on MPI processes and keeping the number of OpenMP threads as one is optimal, producing the fastest runtime for both 660x120 (3.58s) and 1320x240 (30.37s). However, as we scale the problem up to 1980x360 and run across multiple nodes, a hybrid approach of using two processes and three threads works better, running in 111.56s on two nodes and 92.86s on four nodes.

The hybrid approach achieves higher throughput GFLOPs than the solely MPI parallelised application, but has lower utilised bandwidth. This is because OpenMP uses shared memory space, so there's less data transfer of halo cells between processes and therefore more time is spent on computation. This phenomenon is also shown by the arithmetic intensity increasing as we increase the OpenMP thread count and decrease the number of MPI processes.

From the roofline model in Figure 2, we can see that the hybrid application remains compute-bound. There is also still a distinct gap between the data points and the roofline, showing that my optimisation with OpenMP could still be improved further to get closer to the theoretical peak performance.

IV. CONCLUSION

In this paper, we successfully parallelised a serial computational fluid dynamics solver into an implementation that's efficient and scalable across distributed memory systems using both MPI and a hybrid approach of MPI and OpenMP. The application parallelised with MPI achieves strong performance and near-linear scalability across problem sizes when run on

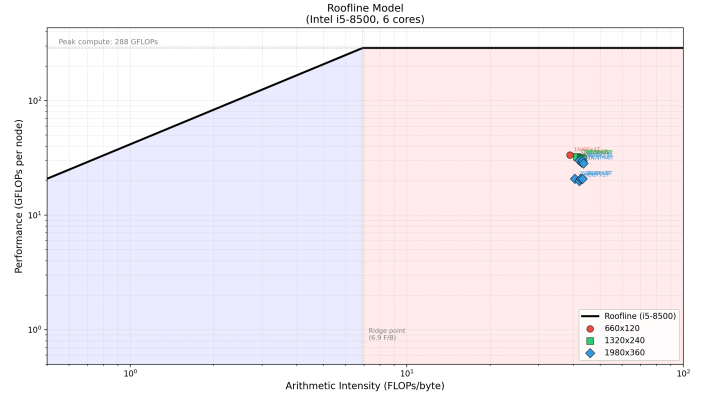


Fig. 2. Roofline model for Karman Parallelised with MPI + OpenMP

a single node, and also shows solid scalability to multi-node configurations, though with diminishing returns as the number of nodes is increased, due to the overhead of communication between nodes. The hybrid approach extends this multi-node scalability by using shared memory parallelism to increase the throughput per node, achieving stronger performance on large problem sizes run across multiple nodes.

It, however, does not resolve the fundamental bottleneck of the communication overhead between nodes, so still exhibits the diminishing returns present with the MPI application. There are also still improvements that can be made to the application in future work, as it was shown that there remains

a large gap between the GFLOPs and bandwidth utilisation achieved and the theoretical peak performance. This future work could include methods for reducing the number of global communications needed, such as by fusing stages, or changing to a 2-dimensional domain decomposition to reduce the number of halo cells that need to be communicated.

REFERENCES

- [1] J. D. Anderson, *Computational Fluid Dynamics: The Basics with Applications*. New York: McGraw-Hill, 1995.
- [2] D. Walker and J. Dongarra, "Mpi: A standard message passing interface," *Supercomputer*, vol. 12, 12 1995.
- [3] L. Dagum and R. Menon, "OpenMP: An industry-standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [4] Intel Corporation, "Intel® Core™ i5-8500 Processor (9M Cache, up to 4.10 GHz) product specifications," <https://www.intel.com/content/www/us/en/products/sku/129939/intel-core-i58500-processor-9m-cache-up-to-4-10-ghz/specifications.html>, 2026.
- [5] Intel, "APP metrics for Intel microprocessors: Intel Core processors," January 2026, revision 8.